

Compte Rendu Livre et compte bancaire POO



Sommaire;

[Sommaire;](#)

[Introduction](#)

[I-Exercice 1 : La Classe Livre](#)

[Résultat Final](#)

[II-Exercice 2 : La Classe Compte](#)

[A. Relation entre Classes \(Client et Compte\)](#)

[B. Encapsulation Avancée et Sécurité](#)

[C. Surcharge de Méthodes et Logique Métier](#)

[Rendu Final](#)

Introduction

Dans ce compte rendu nous allons détailler des solutions et des concepts mis en œuvre dans le cadre des exercices pratiques portant sur la Programmation Orientée Objet (POO). Ces exercices nous permettent d'acquérir les bases sur les programmations des classes, de l'encapsulation, des constructeurs, et des relations entre objets.

Ce compte rendu comprendra deux principaux exercices:

L'Exercice 1 (La Classe Livre) : Centré sur la définition simple d'une classe, l'initialisation des attributs et l'affichage des données.

L'Exercice 2 (La Classe Compte) : Centré sur l'approfondissement des concepts en introduisant la composition de classes (Client/Compte), la lecture seule, la surcharge de méthodes, et l'utilisation des opérations bancaires.

I-Exercice 1 : La Classe Livre

L'objectif de cet exercice est de comprendre la création d'une classe simple et l'utilisation de la lecture seule(encapsulation).

Les principales notions que nous allons utilisés sont:

- **Classe et Attributs** : Définition de la classe **Livre** pour modéliser une entité réelle en mémoire, avec les attributs de base : Titre, Auteur, et Prix.

```
import java.util.Scanner;
public class Exo1 {
    static class Livre {
        // Attributs
        private String titre;
        private String auteur;
        private double prix;
```

- **Constructeur** : Mise en place d'une méthode spéciale (constructeur) pour initialiser les attributs dès la création de l'objet Livre.

```
// Constructeur
public Livre(String titre, String auteur, double prix) {
    this.titre = titre;
    this.auteur = auteur;
    this.prix = prix;
```

- **Propriétés (Getters/Setters)** : Utilisation des propriétés pour accéder aux attributs de manière contrôlée. C'est le pilier de la lecture seule, garantissant que les données internes de l'objet ne sont pas manipulées directement.
 - **Getters** (lecture) pour récupérer les valeurs.

```
// Méthodes d'accès (getters)
public String getTitre() {
    return titre;
}

public String getAuteur() {
    return auteur;
}

public double getPrix() {
    return prix;
}
```

- **Setters** (écriture) pour modifier les valeurs.

```
// Méthodes de modification (setters)
public void setTitre(String titre) {
    this.titre = titre;
}

public void setAuteur(String auteur) {
    this.auteur = auteur;
}

public void setPrix(double prix) {
    this.prix = prix;
}
```

- **Méthode d'Affichage** : Définition de la méthode `Afficher()` qui regroupe la logique d'affichage des informations, permettant à l'objet de présenter ses propres données de façon structurée.

```
// Méthode Afficher
public void afficher() {
    System.out.println("Titre: " + titre + ", Auteur: " + auteur + ", Prix: " + prix);
}
```

Ensuite nous achevons le code avec les anciennes connaissances acquises.

```

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    // Test du Livre 1
    System.out.println("Livre 1:");
    System.out.print("Donner le titre: ");
    String titre1 = scanner.nextLine();

    System.out.print("Donner l'auteur: ");
    String auteur1 = scanner.nextLine();

    System.out.print("Donner le prix: ");
    double prix1 = scanner.nextDouble();
    scanner.nextLine(); // Pour consommer le retour à la ligne
    // Création du premier livre
    Livre livre1 = new Livre(titre1, auteur1, prix1);

    // Affichage des informations avec les getters
    System.out.println("Le titre est " + livre1.getTitre());
    System.out.println("L'auteur est " + livre1.getAuteur());
    System.out.println("Le prix est " + livre1.getPrix());
}

```

Résultat Final

```

Livre 1:
Donner le titre: Dorian Gray
Donner l'auteur: Jean paul
Donner le prix: 20
Le titre est Dorian Gray
L'auteur est Jean paul
Le prix est 20.0
Titre: Dorian Gray, Auteur: Jean paul, Prix: 20.0

Livre 2:
Donner le titre: De la haine a l'amour
Donner l'auteur: Lamiss
Donner le prix: 100
Titre: De la haine a l'amour, Auteur: Lamiss, Prix: 100.0

```

II-Exercice 2 : La Classe Compte

Dans cet exercice, nous avons étudié la relation entre classe et l'importance de la lecture seule et sécuritaire pour la gestion des opérations financières.

Dans cet objectif nous avons utilisés comme notions principales:

A. Relation entre Classes (Client et Compte)

Composition : La classe Compte est liée à la classe Client (propriétaire). On dit qu'un Compte "à un" Client. La classe Compte contient donc un attribut de type Client.

```
import java.util.Scanner;
public class Exo2 {
    // Classe Client
    static class Client {
        // Attributs
        private String CIN;
        private String nom;
        private String prenom;
        private String tel;

        // Constructeur avec tous les attributs
        public Client(String CIN, String nom, String prenom, String tel) {
            this.CIN = CIN;
            this.nom = nom;
            this.prenom = prenom;
            this.tel = tel;
        }

        // Constructeur avec CIN, nom et prénom seulement
        public Client(String CIN, String nom, String prenom) {
            this.CIN = CIN;
            this.nom = nom;
            this.prenom = prenom;
            this.tel = "";
        }
    }
}
```

B. Encapsulation Avancée et Sécurité

- **Propriétés en Lecture Seule** : Les attributs "Numéro de Compte" et "Solde" ont été définis avec des propriétés en lecture seule (seulement des **Getters**).
 - Cette restriction est importante car elle empêche le programmeur de modifier le solde ou le numéro directement.
 - Les modifications du solde doivent obligatoirement passer par les méthodes d'opération dédiées (Créditer() et Débiter()).

```
// Getters
public String getCIN() { return CIN; }
public String getNom() { return nom; }
public String getPrenom() { return prenom; }
public String getTel() { return tel; }

// Setters
public void setCIN(String CIN) { this.CIN = CIN; }
public void setNom(String nom) { this.nom = nom; }
public void setPrenom(String prenom) { this.prenom = prenom; }
public void setTel(String tel) { this.tel = tel; }

// Méthode Afficher
public void afficher() {
    System.out.println("CIN: " + CIN);
    System.out.println("NOM: " + nom);
    System.out.println("Prénom: " + prenom);
    System.out.println("Tél: " + tel);
}
```

C. Surcharge de Méthodes et Logique Métier

- **Surcharge (Overloading) :** Les méthodes Créditer() et Débiter() sont **surchargées**. Cela signifie qu'elles portent le même nom, mais gèrent différentes situations en fonction du nombre de paramètres.
 - **Opération Simple :** Créditer(somme) (Dépôt) et Débiter(somme) (Retrait).
 - **Opération de Transfert :** Créditer(somme, compte_source) et Débiter(somme, compte_dest) (Transfert de compte à compte).

```
// Méthodes Créditer
public void crediter(double montant) {
    if (montant > 0) {
        solde += montant;
    }
}

public void crediter(double montant, Compte compteSource) {
    if (montant > 0 && compteSource.solde >= montant) {
        compteSource.solde -= montant;
        this.solde += montant;
    }
}

// Méthodes Débiter
public boolean debiter(double montant) {
    if (montant > 0 && solde >= montant) {
        solde -= montant;
        return true;
    }
    return false;
}

public boolean debiter(double montant, Compte compteDestination) {
    if (this.debiter(montant)) {
        compteDestination.crediter(montant);
        return true;
    }
}
```

- **Contrôle de Gestion :** La méthode Débiter() doit impérativement intégrer une vérification pour s'assurer que le solde est suffisant avant d'effectuer le retrait ou le transfert, simulant une règle métier essentielle en banque.

```
// Méthodes Débiter
public boolean debiter(double montant) {
    if (montant > 0 && solde >= montant) {
        solde -= montant;
        return true;
    }
    return false;
}
```

Rendu Final

```
Compte 1:
Donner Le CIN: 545665
Donner Le Nom: dfftfyu
Donner Le Prénom: tydtfyu
Donner Le numéro de téléphone: 54458

--- Affichage initial ---
Numéro de Compte: 1
Solde de compte: 0.0
Propriétaire du compte :
CIN: 545665
NOM: dfftfyu
Prénom: tydtfyu
Tél: 54458

Donner le montant à déposer: 1000
```

```
--- Après dépôt de 1000.0 ---
Numéro de Compte: 1
Solde de compte: 1000.0
Propriétaire du compte :
CIN: 545665
NOM: dfftfyu
Prénom: tydtfyu
Tél: 54458

Donner le montant à retirer: 100
Opération bien effectuée

--- Situation finale ---
Numéro de Compte: 1
Solde de compte: 900.0
Propriétaire du compte :
CIN: 545665
NOM: dfftfyu
Prénom: tydtfyu
Tél: 54458
```